

H0005 — Generar llaves

Sistema de Gestión de Torneos Deportivos de Contacto (Taekwondo) · Funcionalidad core

Épica: Inscripción y llaves

13 puntos

KANO: Atractivo 

Generar/Ganador: Coordinador

1. Qué se implementó

El corazón del sistema: a partir de los competidores cargados, el Coordinador **genera las llaves** de eliminación directa. Incluye tres capacidades: (1) **clasificar** a los competidores en las categorías que encajan, (2) **generar el bracket** de cada categoría, y (3) **registrar ganadores** que avanzan de ronda, con actualización **en vivo** del bracket para todos los espectadores. La visualización usa **React Flow**.

2. El algoritmo del bracket (core)

`BracketGeneratorService` es una función pura que, dados los competidores de una categoría, produce las `LlaveCompetencia` (matches):

- Tamaño del bracket = siguiente potencia de 2 $\geq n$; rondas = $\log_2(\text{tamaño})$; BYEs = tamaño - n.
- Competidores mezclados aleatoriamente (sin seeding).
- BYEs distribuidos **uniformemente** en la ronda 1 (no quedan juntos) — cada BYE pasa directo.
- Matches vacíos para las rondas siguientes; los ganadores de BYE se propagan automáticamente.
- 1 competidor = campeón directo (0 matches); 0 = error.

Tests obligatorios del skill: 10/10 en verde. Cubren n = 2, 3, 4, 6, 8 (rondas, matches y BYEs exactos), 1 competidor (campeón, 0 matches), 0 (excepción), BYEs no adyacentes, todos los competidores ubicados, y propagación del BYE a la ronda siguiente.

Clasificación competidor → categoría

`ClasificadorCompetidores.Encaja` compara los atributos del competidor (sexo, edad, peso, graduación) contra los rangos de la categoría (con extremos abiertos y peso opcional). Al generar, cada competidor *sin categoría* se asigna a la **primera categoría que encaja** (first-fit).

3. Backend (Clean Architecture + CQRS)

Método	Ruta	Rol	Qué hace
POST	<code>/torneos/{id}/llaves/generar</code>	Coordinador	Clasifica competidores y genera brackets (2+ por categoría). Devuelve un resumen.
GET	<code>/torneos/{id}/llaves/{categoriaId}</code>	Todos	Bracket de una categoría (matches agrupados por ronda).
PUT	<code>/torneos/{id}/llaves/{llaveId}/ganador</code>	Coordinador	Registra el ganador, lo avanza a la ronda siguiente y notifica por SignalR.

Nuevos: `Categoria.LlavesGeneradas` (flag + migración), `LlaveCompetenciaRepository`, `ClasificadorCompetidores`, `BracketGeneratorService`, y los DTOs del bracket.

```
POST /torneos/{id}/llaves/generar
-> Clasifica competidores sin categoria (first-fit por atributos)
-> Por cada categoria con 2+ y sin llaves: BracketGeneratorService.Generar
    -> persiste llaves + marca Categoria.LlavesGeneradas = true
-> Resumen: por categoria (generado / motivo) + competidores sin clasificar
```

Reglas de negocio

- **No regenerar:** una categoría con `LlavesGeneradas` no vuelve a generarse (regla 6 del skill).
- **Categoría bloqueada:** con llaves generadas no se puede editar ni eliminar (guard en backend + oculto en UI).
- Registrar ganador: valida que el ganador sea uno de los dos competidores del match (409 si no).
- Todo valida pertenencia al torneo (404) para no operar sobre datos de otro torneo.

4. Frontend (React Flow + SignalR)

Archivo	Rol
<code>components/bracket/bracketLayout.ts</code>	Convierte el Bracket en nodos/aristas; el espaciado vertical se duplica por ronda.
<code>components/bracket/MatchNode.tsx</code>	Nodo custom: dos competidores (o BYE "pasa directo"); el ganador se resalta en verde con ✓.
<code>components/bracket/BracketView.tsx</code>	ReactFlow con pan + zoom (Controls), izquierda → derecha, fitView.
<code>components/bracket/GenerarLlavesButton.tsx</code>	CTA que dispara la generación y muestra el resumen en un toast.
<code>lib/signalr.ts + hooks/useBracket.ts</code>	Conexión al hub y hooks (useBracket, useGenerarLlaves, useRegistrarGanador).
<code>routes/.../categorias/\$categoriaId/llaves.tsx</code>	Página del bracket con actualización en vivo.

Registrar ganador (Coordinador): hace clic en un competidor de un match → se marca ganador y avanza. El backend emite `MatchActualizado` por SignalR y *todos* los clientes del torneo refrescan el bracket automáticamente (invalidando la query).

5. Decisiones y límites (para conversar)

Clasificación first-fit. Un competidor puede encajar en varias categorías (rangos que se solapan, o Combate y Formas a la vez). Por ahora va a la primera que encaja. Para ajustar, el Coordinador edita categorías/competidores *antes* de generar (el CRUD lo permite).

Sin regeneración. Una vez generadas, las llaves no se rehacen (regla del skill). Si la clasificación quedó mal, hoy no hay "borrar llaves". Es un candidato claro a una historia futura (des-generar / re-clasificar).

Estado del torneo. No hay transición Borrador→Activo→Finalizado todavía; generar llaves no cambia el estado. Los guards de "Finalizado" quedan listos para cuando exista esa historia.

Multi-categoría (Combate + Formas). El competidor tiene un único `categoria_id`, así que hoy compite en una sola. Modelar la doble participación es una extensión futura.

6. Pruebas y verificación

- `BracketGeneratorServiceTests` — 10 casos obligatorios del skill.
- `GenerarLlavesCommandHandlerTests` — clasifica + genera, 1 competidor no genera, no-encaje queda sin clasificar, no regenera, torneo inexistente.
- `RegistrarGanadorCommandHandlerTests` — registra + propaga + notifica, ganador inválido (409), match inexistente / de otro torneo (404).
- + guards de categoría bloqueada por llaves.

- ✓ Backend compila 0 errores · **90/90 tests** en verde (+20).
- ✓ Frontend `tsc + vite build` sin errores (React Flow + SignalR).

Nota: verificación end-to-end pendiente de reiniciar la API (build en ejecución es anterior). Al arrancar se aplican las migraciones nuevas (sexo/categoría nullable y `llaves_generadas`).

7. Checklist

Algoritmo de bracket + 10 tests obligatorios	✓
Clasificación por atributos (sexo/edad/peso/graduación)	✓
Endpoints generar / ver / registrar ganador + Roles	✓
Repositorio de llaves + flag <code>LlavesGeneradas</code> + migración	✓
SignalR: notifier (Application ← API) + evento <code>MatchActualizado</code>	✓
React Flow: layout + <code>MatchNode</code> + <code>BracketView</code> (pan/zoom, BYEs, ganador ✓)	✓
Registrar ganador por clic (solo Coordinador) + actualización en vivo	✓
Categoría con llaves: bloqueada para editar/eliminar (backend + UI)	✓
Comentarios XML <code><summary></code> en el código nuevo	✓
Verificación E2E	pendiente (reiniciar API)